NAME : S.KARTHIKEYAN 3122 22 5001 056
CSE B

UCS 2312 Data Structures Lab

Exercise 12: Implementation of Hash Table using Closed and Open addressing methods

[CO2, K3]

The HashTableADT contains hash table and its size. Hash function to be used for the insertion of elements is *x mod tableSize*. Use Separate chaining method to resolve the collision.

- **void init(HashTableADT *H) – To initialize the size of Hash Table**
- **void insertElementL (HashTableADT *H, int x)– To insert the input key into the hash table**
- **int searchElement(HashTableADT *H, int key) – Searching an element in the hash table, if found return 1, otherwise return -1**
- **void displayHT(HashTableADT *H) – Display the elements in the hash table**

1. Demonstrate ADT with the following test case
insert 23, 45, 69, 87, 48, 67, 54, 66, 53

Contents of Hash Table
(Separate Chaining)
0 :
1 :
2 :
3 : 23 → 53
4 : 54
5 : 45
6 : 66
7 : 87 → 67
8 : 48
9 : 69

2. Create another hash table ADT with following functions for open addressing methods, namely, Quadratic probing and Double Hashing.

- **void insertElementL (HashTableADT *H, int x)– To insert the input key into the hash table**
- **void displayHT(HashTableADT *H) – Display the elements in the hash table**

Note: For Double hashing, the second hash function is 7-(x%7)

NAME : S.KARTHIKEYAN 3122 22 5001 056
CSE B

Demonstrate the ADT with the following testcase
(i) **Contents of Hash Table (Quadratic Probing)**
0 →
1 →67
2 →53
3 →23
4 →54
5 →45
6 →66
7 →87
8 →48
9 →69

(ii) **Contents of Hash Table (Double Hashing)**
0 →67
1 →
2 →53
3 →23
4 →54
5 →45
6 →66
7 →87
8 →48
9 →69

1. Separate Chaining

Code:

ADT File:

```
# include <stdio.h> # include
<stdlib.h>

struct node
{
        int data;
        struct node * next;
};

struct hashTable
{
        int size;
```

```
        struct node * list[100];
};

void initHashTable (struct hashTable * H, int size)
{
        H->size=size;
        for (int i=0;i<H->size;i++)
        {
        H->list[i]= (struct node *)malloc(sizeof(struct node)); H->list[i]->next= NULL;
        }
}

void insertElement (struct hashTable * H, int x)
{
        struct node * temp;
        temp= (struct node *)malloc(sizeof(struct node)); temp->next=NULL;
        temp->data=x; int
        h=x%H->size;
        struct node * ptr; ptr= H-
        >list[h];
        while (ptr->next !=NULL)
        {
                ptr=ptr->next;
        }
        ptr->next=temp;
}

int searchElement(struct hashTable * H, int key)
{
        int  h=key%H->size;  struct
        node * ptr; ptr= H->list[h];
        while (ptr->next !=NULL)
        {
                ptr=ptr->next;
                if (ptr->data==key)
                {
                        return 1;
                }
        }
        return -1;
}

void displayHT (struct hashTable * H)
{
        for (int i=0; i<H->size;i++)
        {   printf("%d  :",i);  struct  node
                *ptr; ptr=H->list[i];
                while (ptr->next!=NULL)
```

```
                {
                        ptr=ptr->next; printf("%d",ptr-
                        >data); if (ptr->next!=NULL)
                                printf("->");
                }
                printf("\n");
        }
}
```

Application file:

```
# include<stdbool.h> # include
"HashADT.h"

void main()
{
        struct hashTable * H= (struct hashTable* ) malloc(sizeof(struct hashTable));
        int choice,size,element;
        printf("Enter size of hash table : "); scanf("%d", &size);
        initHashTable(H,size);
        while (true)
        {
                printf("----- HASH TABLE OPERATIONS---------- \n1.
                Insert\n2. Search\n3. Display\n4. Exit\nEnter Choice:
                \n");
                scanf("%d",&choice); if
                (choice==1)
                {
                        printf("Enter element: "); scanf ("%d",
                        &element); insertElement(H,element);
                }
                else if (choice==2)
                {
                        printf("Element to be searched: "); scanf ("%d",
                        &element);
                        if(searchElement(H,element)==-1)
                        {
                   printf("%d is not present in hash table\n ",element);
                        }
                        else
                        printf("%d is present in hash table\n",element);
                }

                else if (choice==3)
                {
                        displayHT(H);
```

```
            }
            else
                  break;
      }
}
```

Output:

```
Enter size of hash table : 10
----- HASH TABLE OPERATIONS -----
1. Insert
2. Search
3. Display
4. Exit
Enter Choice:
1
Enter element: 23
----- HASH TABLE OPERATIONS -----
1. Insert
2. Search
3. Display
4. Exit
Enter Choice:
1
Enter element: 45
----- HASH TABLE OPERATIONS -----
1. Insert
2. Search
3. Display
4. Exit
Enter Choice:
1
Enter element: 69
----- HASH TABLE OPERATIONS -----
1. Insert
2. Search
3. Display
4. Exit
Enter Choice:
1
Enter element: 87
----- HASH TABLE OPERATIONS -----
1. Insert
2. Search
3. Display
4. Exit
Enter Choice:
1
Enter element: 48
```

```
----- HASH TABLE OPERATIONS -----
1. Insert
2. Search
3. Display
4. Exit
Enter Choice:
1
Enter element: 67
----- HASH TABLE OPERATIONS -----
1. Insert
2. Search
3. Display
4. Exit
Enter Choice:
1
Enter element: 54
----- HASH TABLE OPERATIONS -----
1. Insert
2. Search
3. Display
4. Exit
Enter Choice:
1
Enter element: 66
----- HASH TABLE OPERATIONS -----
1. Insert
2. Search
3. Display
4. Exit
Enter Choice:

1
Enter element: 53
----- HASH TABLE OPERATIONS -----
1. Insert
2. Search
3. Display
4. Exit
Enter Choice:
3
0 :
1 :
2 :
3 :23->53
4 :54
5 :45
6 :66
7 :87->67
8 :48
9 :69
```

2. Additional Routines:

Quadratic Probing:

NAME : S.KARTHIKEYAN 3122 22 5001 056
CSE B

Code:

```
#include <stdio.h> #include
<stdlib.h> #include<stdbool.h>

struct hashTable
{
int size;
int table[100];
};
void initHashTable (struct hashTable *H, int size)
{
        H->size = size;
        for (int i=0;i< H->size; i++)
        {
        H->table[i]=-1;
        }
}
void insertElement (struct hashTable *H, int x)
{
        int pos,flag=1;
        for(int i=0;i<H->size;i++)
        {
                pos=(x+(i*i))%(H->size); if(H-
                >table[pos]==-1)
                {
                        H->table[pos]=x; flag=0;
                        break;
                }
        }
        if(flag)
        {
        printf("Hash Table is full.\n");
        }
}

void displayHT (struct hashTable * H)
{
          for (int i=0; i<H->size;i++)
          {         printf("%d :",i);
                    if (H->table[i]!=-1)
                    {
                              printf("%d", H->table[i]);
                    }
                    printf("\n");
          }
}

void main()
```

NAME : S.KARTHIKEYAN 3122 22 5001 056
CSE B

```
{
        struct hashTable * H= (struct hashTable*
 )malloc(sizeof(structhashTable)); int
           choice,size,element;
           printf("Enter size of hash table : "); scanf("%d", &size);
           initHashTable(H,size);
           while (true)
           {
                    printf("----- HASH TABLE OPERATIONS---------- \n1.
 Insert\n2. Display\n3. Exit\nEnter Choice: \n"); scanf("%d", &choice);
                    if (choice==1)
                    {
                             printf("Enter element: "); scanf ("%d",
                             &element); insertElement(H,element);
                    }

                    else if (choice==2)
                    {
                             displayHT(H);
                    }
                    else
                   break;
         }
 }
```

NAME : S.KARTHIKEYAN 3122 22 5001 056
CSE B

Output :

```
Enter size of hash table : 10
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 23
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 45
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 69
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 87
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 48
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 67
```

```
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 54
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 66
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 53
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
2
0 :
1 :67
2 :53
3 :23
4 :54
5 :45
6 :66
7 :87
8 :48
9 :69
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
3
```

3. Double Hashing

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include<stdbool.h>

struct hashTable
{
	int size;
	int table[100];
};

void initHashTable (struct hashTable *H, int size)
{
	H->size = size;
	for (int i=0;i< H->size; i++)
	{
		H->table[i]=-1;
	}
}

bool isPrime(int num) { if (num <= 1)
	{
		return false;
	}
	for (int i = 2; i * i <= num; ++i) { if (num % i == 0) {
			return false;
		}
	}
	return true;
}


int greatestPrime(struct hashTable * H) { for (int i = H->size- 1; i >
	1; --i) {
		if (isPrime(i)) { return i;
		}
	}
	return -1;
}

void insertElement (struct hashTable *H, int key)
{
	int index,i;
	int hash = key % H->size;
	int hash2 = greatestPrime(H) - (key % greatestPrime(H)); for (i = 0; i < H->size; i++) {
```

```
            index = (hash + i * hash2) % H->size; if (H->table[index]
            == -1) {
                  H->table[index] = key; break;
            }
      }
}
            index = (hash + i * hash2) % H->size; if (H->table[index]
            == -1) {
                  H->table[index] = key; break;
            }
      }
}


void displayHT (struct hashTable *H)
{

        for (int i=0; i<H->size;i++)
        {       printf("%d :",i);
                if (H->table[i]!=-1)
                {
                        printf("%d", H->table[i]);
                }
                printf("\n");
        }
}

void main()
{
      struct hashTable * H= (struct hashTable*
)malloc(sizeof(struct hashTable)); int
        choice,size,element;
        printf("Enter size of hash table : "); scanf("%d", &size);
        initHashTable(H,size);
        while (true)
        {
                printf("----- HASH TABLE OPERATIONS ---------- \n1.
Insert\n2. Display\n3. Exit\nEnter Choice: \n"); scanf("%d", &choice);
                if (choice==1)
                {
                        printf("Enter element: "); scanf ("%d",
                        &element); insertElement(H,element);
                }

                else if (choice==2)
                {
                        displayHT(H);
                }
                Else
                break;
      }
}
```

NAME : S.KARTHIKEYAN 3122 22 5001 056
CSE B

Output:

```
Enter size of hash table : 10
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 23
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 45
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 69
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 87
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 48
```

```
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 67
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 54
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 66
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
1
Enter element: 53
----- HASH TABLE OPERATIONS -----
1. Insert
2. Display
3. Exit
Enter Choice:
2
0 :67
1 :
2 :53
3 :23
4 :54
5 :45
6 :66
7 :87
8 :48
9 :69
```

LEARNING OUTCOME:

After the completion of this exercise, I have learnt hashing techniques.

NAME : S.KARTHIKEYAN 3122 22 5001 056

CSE B